HTML FOUNDATIONS

Step-by-Step Lecture Notes

Semantic HTML • Tags and Attributes • Accessibility • Document Structure • Metadata

Prepared from the uploaded presentation:
CO1_2. HTML basics, Tags.pptx

How to Use These Notes

These notes are arranged for classroom delivery. Teach the ideas in sequence, type the examples in a code editor, save each example with the .html extension, and open it in a browser. The explanations expand the brief examples in the source presentation while retaining its topics.

Learning Outcomes

- Explain what HTML is and how tags, elements, and attributes work.
- Create a valid HTML5 document with metadata and visible page content.
- Choose semantic elements for page structure and meaningful content.
- Create headings, text, links, images, lists, tables, embedded media, and forms.
- Apply basic accessibility practices, keyboard navigation, and ARIA appropriately.
- Recognize obsolete frame elements and use modern alternatives.

Contents / Teaching Sequence

1. 1. Introduction to HTML and tag syntax
2. 2. The HTML5 document skeleton
3. 3. Headings, paragraphs, and text semantics
4. 4. Links and navigation
5. 5. Images, figures, and responsive image sources
6. 6. Lists
7. 7. Tables for tabular data
8. 8. Semantic page landmarks and grouping elements
9. 9. Generic containers: div and span
10. 10. Iframes and embedded content
11. 11. Forms and form controls
12. 12. Audio, video, track, details, and summary
13. 13. Preformatted text, code, buttons, and line/thematic breaks
14. 14. Accessibility and ARIA
15. 15. Keyboard navigation and inclusive design
16. 16. Metadata, SEO, and structured data
17. 17. Integrated practical exercise and revision checklist

1. Introduction to HTML

HTML stands for HyperText Markup Language. It is the standard markup language used to describe the structure and meaning of content on a web page. HTML is not a programming language: it marks content so browsers can interpret headings, paragraphs, links, images, forms, and other elements.

1.1 What is a tag?

A tag is markup written inside angle brackets. Most elements have an opening tag, content, and a closing tag. The closing tag includes a forward slash.

```
<p>This is a paragraph.</p>
```

Here, is the opening tag, the sentence is the content, and is the closing tag.

1.2 Element versus tag

- Tag: the markup token, such as or .
- Element: the complete structure, such as Hello .
- Attribute: extra information written in the opening tag, usually as name="value".

```
<a href="https://example.com">Visit Example</a>
```

In this example, href is an attribute and its value is the destination URL.

1.3 Paired and void elements

- Paired elements have opening and closing tags: , <h1>, , <a>, <button>.
- Void elements do not wrap content and do not have closing tags: <img>,
, <hr>, <meta>, <link>, <input>.

1.4 Nesting rules

```
<p>This is <strong>important</strong> information.</p>
```

Close nested elements in the reverse order in which they were opened. Avoid overlapping tags.

Teaching note: Ask students to identify the opening tag, closing tag, content, and attributes in three sample elements.

2. The HTML5 Document Skeleton

Every ordinary HTML page should begin with a document declaration and contain html, head, and body elements.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>My First Web Page</title>
</head>
<body>
  <h1>Welcome to HTML</h1>
  <p>This is my first web page.</p>
</body>
</html>
```

Tag / element	Purpose and usage
<!DOCTYPE html>	Declares an HTML5 document and asks the browser to use standards mode. It is a declaration, not an HTML element.
<html>	Root element; all page markup is contained inside it. The lang attribute identifies the main language.
<head>	Contains document information and resource

	references; most of its contents are not displayed as page content.
<title>	Sets the browser-tab title and commonly supplies the bookmark/search title.
<body>	Contains the content displayed in the browser window.
<meta>	Supplies metadata such as character encoding, viewport settings, and description. It is a void element.

2.1 Important document attributes

- lang="en" on <html> identifies the primary language; use an appropriate language code, such as "en" or "te".
- charset="utf-8" supports a broad range of characters and should appear near the beginning of the head.
- name="viewport" with content="width=device-width, initial-scale=1" helps pages adapt to mobile screen widths.

2.2 Save and run

18. Open a text editor or VS Code.
19. Create a file named index.html.
20. Type the complete skeleton.
21. Save the file.
22. Open it in a web browser and inspect the tab title and visible page content.

3. Headings, Paragraphs, and Text Semantics

3.1 Heading elements: h1 to h6

```
<h1>Web Development</h1>
<h2>HTML</h2>
<h3>HTML Elements</h3>
<h4>Text Elements</h4>
<h5>Inline Semantics</h5>
<h6>Additional Notes</h6>
```

The six heading levels express document hierarchy. Use one clear main page heading in ordinary page designs, then organize subsections with h2, h3, and so on. Do not choose a heading level merely to obtain a particular font size; use CSS for visual sizing.

3.2 Paragraph and inline text elements

Tag / element	Purpose and usage
<p>	Defines a paragraph of text.

****	Marks content as strongly important; browsers commonly render it bold.
****	Adds stress emphasis; browsers commonly render it italic.
<b>	Draws attention stylistically without adding the same importance meaning as strong.
<i>	Marks text in an alternate voice or mood, or a conventional italicized span; it is not a substitute for em in every case.
<blockquote>	Represents an extended quotation from another source.
** **	Creates a line break within text, such as an address or poem; do not use it to create page spacing.
<hr>	Represents a thematic break between paragraph-level topics; commonly displayed as a horizontal rule.

```
<h1>HTML Basics</h1>
<p>HTML gives a page its <strong>structure</strong>.</p>
<p>You can add <em>emphasis</em> to a word.</p>
<p><b>Notice:</b> b is stylistic; strong conveys importance.</p>
<blockquote>
  The web is built on open standards.
</blockquote>
<p>Department of Computer Science<br>KL University</p>
<hr>
<p>Next topic: hyperlinks.</p>
```

3.3 Common mistakes

- Using heading tags only to make text large.
- Skipping heading levels without a structural reason.
- Using repeated
 tags to create vertical spacing instead of CSS.
- Using <b> or <i> when the intended meaning is importance or emphasis.

4. Hyperlinks: The Anchor Element

The <a> element creates a hyperlink. Its href attribute identifies the destination. Links navigate; buttons perform actions.

```
<!-- Absolute link -->
<a href="https://example.com">Visit Example</a>

<!-- Relative link -->
<a href="/docs/intro.html">Read the introduction</a>

<!-- Same-page jump -->
<a href="#contact">Go to contact details</a>

<!-- Email and telephone links -->
<a href="mailto:info@example.com">Email us</a>
<a href="tel:+911234567890">Call us</a>

<!-- Download link -->
<a href="report.pdf" download>Download the report</a>

<!-- External page in a new tab -->
<a href="https://example.com"
  target="_blank"
  rel="noopener noreferrer">Open Example in a new tab</a>
```

Tag / element	Purpose and usage
href	Destination URL, file, email address, phone number, or fragment identifier.
target	Controls where the destination opens; target="_blank" commonly opens a new tab/window.
rel	Describes the relationship to the destination; noopener prevents the new page from controlling the opener, and noreferrer suppresses referrer information.
download	Suggests downloading the linked resource rather than navigating to it.

4.1 Good link text

- Good: Read the HTML accessibility guide.
- Avoid vague text such as "Click here" when the destination is not clear from context.
- Tell users when a link opens a new tab if that behaviour may surprise them.

4.2 Link versus button

- Use <a> when the user moves to another page, file, email address, phone number, or page section.
- Use <button> when the user triggers an action such as submitting, opening, toggling, or deleting.

5. Images, Figures, and Responsive Image Sources

5.1 The img element

```
<img src="mountain.png"
   alt="Snowy mountain peak at sunrise"
   width="600"
   height="400"
   loading="lazy">
```

Tag / element	Purpose and usage
src	Path or URL of the image file.
alt	Text alternative describing the image's purpose or information. Use alt="" for purely decorative images.
width / height	Provide intrinsic dimensions to help the browser reserve space and reduce layout shift.
loading	loading="lazy" defers loading until the image is near the viewport; useful for off-screen images.

The <img> element is a void element. It does not contain child text and has no closing tag.

5.2 Figure and figcaption

```
<figure>
  <img src="chart.png" alt="Sales increased from Q1 to Q3">
  <figcaption>Figure 1: Quarterly sales trend</figcaption>
</figure>
```

- <figure> groups self-contained media or illustrative content.
- <figcaption> supplies the caption for its figure.

5.3 Picture and source

```
<picture>
  <source srcset="banner.webp" type="image/webp">
  <img src="banner.jpg" alt="Students working in a computer laboratory">
</picture>
```

- <picture> provides a container for alternative image sources.
- <source> specifies a candidate resource; inside picture, srcset supplies the image candidate and type can identify its media type.
- The nested <img> is the fallback and should still have meaningful alt text.

The source presentation introduces picture/source and srcset. Advanced responsive srcset and sizes selection can be taught as an extension.

6. Lists: ul, ol, dl

6.1 Unordered list

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>
```

 creates a bulleted list. Each list item is represented by <li>.

6.2 Ordered list

```
<ol start="1">
  <li>Plan the page</li>
  <li>Write the HTML</li>
  <li>Test in a browser</li>
</ol>
```

 creates a sequential list. The start attribute sets the starting number.

6.3 Description list

```
<dl>
  <dt>HTML</dt>
  <dd>Defines the structure of a web page.</dd>
</dl>
```

<dl> contains terms and descriptions. <dt> marks a term/name; <dd> supplies its description.

6.4 Nested lists

```
<ul>
  <li>Frontend
    <ul>
      <li>HTML</li>
      <li>CSS</li>
    </ul>
  </li>
  <li>Backend</li>
</ul>
```

Teaching note: Ask learners to create a list of five subjects, a numbered procedure for logging into a portal, and a glossary with three terms.

7. Tables: Data, Not Page Layout

Use a table when information has a meaningful row-and-column relationship, such as marks, timetables, or sales figures. Do not use tables to position page sections; use CSS layout tools.

```
<table>
  <caption>Quarterly sales</caption>
  <thead>
    <tr>
      <th scope="col">Quarter</th>
      <th scope="col">Sales</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">Q1</th>
      <td>40</td>
    </tr>
    <tr>
      <th scope="row">Q2</th>
      <td>55</td>
    </tr>
  </tbody>
</table>
```

Tag / element	Purpose and usage
<table>	Creates the table.
<caption>	Provides a title/description for the table, useful to assistive technology.
<thead>	Groups header rows.
<tbody>	Groups the main data rows.
<tfoot>	Groups footer rows, such as totals, when needed.
<tr>	Creates a table row.
<th>	Creates a header cell.
<td>	Creates a data cell.
scope	On th, identifies whether the header applies to a row (scope="row") or column (scope="col").
colspan	Makes a cell span multiple columns.
rowspan	Makes a cell span multiple rows.

7.1 Table example with merged cells

```
<table>
  <caption>Workshop schedule</caption>
  <tr>
    <th scope="col">Day</th>
    <th scope="col">Session</th>
    <th scope="col">Room</th>
  </tr>
  <tr>
    <th scope="row">Monday</th>
    <td colspan="2">HTML practical laboratory</td>
  </tr>
</table>
```

Use colspan and rowspan only when the data relationship genuinely requires merged cells. Complex tables need careful header associations.

8. Semantic HTML5: Page Structure and Landmarks

Semantic HTML means choosing elements for their meaning rather than using generic containers for everything. Semantic landmarks help browsers, search engines, and screen-reader users understand and navigate a page.

```
<header>
  <a href="/">University Portal</a>
  <nav aria-label="Primary">
    <a href="/home">Home</a>
    <a href="/courses">Courses</a>
  </nav>
</header>

<main>
  <article>
    <h1>Introduction to HTML</h1>
    <p>HTML describes web-page structure.</p>
  </article>
  <aside>
    <h2>Related resources</h2>
    <a href="/resources">Study resources</a>
  </aside>
</main>

<footer>
  <p>Contact: info@example.edu</p>
</footer>
```

Tag / element	Purpose and usage
<header>	Introductory content for a page or section, such

	as branding, title, or introductory links.
<nav>	A major navigation block. Use for important groups of navigation links.
<main>	The dominant content of the document. Normally one visible main element per page.
<article>	A self-contained composition that could stand independently, such as a blog post or news item.
<section>	A thematic grouping of content, usually with a heading, such as a chapter.
<aside>	Related or tangential content, such as a sidebar or related links.
<footer>	Footer information for a page or section, such as authorship, legal details, or contact information.
** **	Generic block container with no inherent semantic meaning; use when no more specific element fits.
<figure>	Self-contained media or illustrative content, often with a figcaption.
<figcaption>	Caption associated with a figure.
<time>	Represents a date/time; datetime can provide a machine-readable value.

8.1 Section, article, and div: how to choose

- Choose <section> for a thematic group, usually with a heading.
- Choose <article> for an independent, reusable or distributable item.
- Choose only as a generic wrapper when no semantic element is appropriate.

```
<article>
  <h2>Student Achievement</h2>
  <p>Our students completed the project.</p>
  <time datetime="2026-11-09">9 November 2026</time>
</article>
```

8.2 Semantic versus div-only markup

```
<!-- Less meaningful structure -->
<div class="top">
  <div class="menu">Home</div>
</div>

<!-- Semantic structure -->
<header>
  <nav aria-label="Primary">
    <a href="/">Home</a>
  </nav>
</header>
```

Semantic elements expose useful roles and landmarks. They can improve keyboard navigation, screen-reader navigation, reader mode, maintainability, and search-engine understanding.

9. Generic Containers: div and span

Tag / element	Purpose and usage
** **	Generic block-level container. Commonly used to group elements for CSS styling or scripting when no semantic element is suitable.
****	Generic inline container. Used to style or identify a small run of text without creating a new block.

```
<div class="student-card">
  <h2>Student Name</h2>
  <p>Department: <span class="department">CSE</span></p>
</div>
```

The class attribute provides a styling or scripting hook. Neither div nor span communicates a specific content role by itself.

Teaching rule: prefer header, nav, main, article, section, aside, footer, figure, or other meaningful elements when they accurately describe the content.

10. Iframes and Embedded Content

An iframe embeds another HTML page or supported web resource within the current page.

```
<iframe
  src="https://www.youtube.com/embed/VIDEO_ID"
  title="Introduction to HTML video"
  width="560"
  height="315"
  loading="lazy"
  sandbox="allow-scripts allow-same-origin"
  allowfullscreen>
</iframe>
```

Tag / element	Purpose and usage
src	URL of the page to embed.
title	Accessible name describing the embedded content; provide a meaningful title.
width / height	Set the iframe dimensions.
loading	loading="lazy" can defer loading until needed.
sandbox	Applies restrictions to the embedded document. Add only the permissions the content needs.
allowfullscreen	Allows fullscreen presentation where supported.

The source presentation notes that the server may block framing through X-Frame-Options or Content Security Policy (CSP). A page cannot override those server-side restrictions.

The old <frame> and <frameset> elements are obsolete and were removed from HTML5. Do not use them in new pages.

11. HTML Forms and Form Controls

Forms collect user input. The form element groups controls and specifies where and how the browser submits the data.

```
<form action="/signup" method="post">
  <fieldset>
    <legend>Account details</legend>

    <label for="name">Name</label>
    <input id="name" name="name" type="text" required>

    <label for="email">Email</label>
    <input id="email" name="email" type="email" required>

    <label for="course">Course</label>
    <select id="course" name="course">
      <option value="html">HTML</option>
      <option value="css">CSS</option>
    </select>

    <label for="message">Message</label>
    <textarea id="message" name="message"></textarea>

    <button type="submit">Send</button>
  </fieldset>
</form>
```

Tag / element	Purpose and usage
<form>	Container for controls; action identifies the submission destination and method commonly uses get or post.
<fieldset>	Groups related form controls.
<legend>	Caption/title for a fieldset.
<label>	Provides an accessible name for a control. Associate using for equal to the control id.
<input>	Creates a control whose behaviour is selected with type; it is a void element.
<select>	Creates a drop-down selection control.
<option>	Defines one selectable choice inside select or a related control.
<textarea>	Creates a multi-line text-entry control.
<button>	Creates a button. Use type=submit for submission, type=button for a normal scripted action, and type=reset only when reset behaviour is intended.

11.1 Important form attributes

- action: destination that receives the submitted form.
- method: submission method. GET appends form data to the URL; POST sends it in the request body.

- id: unique identifier, commonly used to connect a label to its control.
- name: key used when the control's value is submitted.
- value: initial or submitted value, depending on control type.
- required: asks the browser to require a value before submission.
- placeholder: short input hint; it does not replace a label.
- autocomplete: describes the kind of information expected, such as autocomplete="email".

11.2 Common input types

Tag / element	Purpose and usage
text	Single-line ordinary text.
email	Email address; browser can perform basic format validation.
password	Masks typed characters on screen.
number	Numeric value; min, max, and step can constrain it.
date	Date picker/control where supported.
checkbox	Independent on/off choice; multiple checkboxes can be selected.
radio	One choice from a group; use the same name for mutually exclusive choices.
file	Allows the user to select a file.
range	Slider for a numeric value within a range.

```
<label for="age">Age</label>
<input id="age" name="age" type="number" min="1" max="100">

<p>Choose a mode:</p>
<label><input type="radio" name="mode" value="online"> Online</label>
<label><input type="radio" name="mode" value="offline"> Offline</label>

<label for="updates">
  <input id="updates" name="updates" type="checkbox" value="yes">
  Send me updates
</label>

<label for="resume">Upload resume</label>
<input id="resume" name="resume" type="file">
```

11.3 Form accessibility rules

- Give every control a visible, meaningful label.
- Ensure each label's for value matches the control's id.
- Do not use placeholder text as the only label.
- Group related controls with fieldset and legend where useful.
- Use suitable input types and required constraints.

- Explain validation errors clearly.

12. Audio, Video, Track, Details, and Summary

12.1 Audio

```
<audio controls>
  <source src="lecture.mp3" type="audio/mpeg">
  Your browser does not support audio playback.
</audio>
```

<audio> embeds sound. The controls attribute asks the browser to display playback controls. A source element supplies a media file and type.

12.2 Video and captions

```
<video controls width="640" poster="preview.jpg">
  <source src="lesson.mp4" type="video/mp4">
  <track src="captions-en.vtt"
       kind="captions"
       srclang="en"
       label="English">
  Your browser does not support video.
</video>
```

Tag / element	Purpose and usage
<video>	Embeds video content. Common attributes include controls, width, height, autoplay, muted, loop, and poster.
<audio>	Embeds audio content. Common attributes include controls, autoplay, muted, and loop.
<source>	Provides alternative media/image resources within media or picture elements.
<track>	Provides timed text such as captions or subtitles for audio/video.

12.3 Details and summary

```
<details>
  <summary>What is HTML?</summary>
  <p>HTML is the markup language used to structure web content.</p>
</details>
```

- <details> creates a disclosure widget that users can open or close.
- <summary> supplies the visible label users activate to reveal or hide the details.

These elements provide native keyboard-operable disclosure behaviour in supporting browsers.

13. Preformatted Text, Code, Buttons, and Break Elements

Tag / element	Purpose and usage
<pre>	Displays preformatted text, preserving whitespace and line breaks.
<code>	Marks a fragment of computer code.
<button>	Creates an interactive button for an action; use a real button instead of a clickable div.
** **	Inserts a line break within text.
<hr>	Marks a thematic break between content topics.

```
<p>Use the <code>console.log()</code> method to print a message.</p>

<pre><code>
function greet() {
  console.log("Hello, HTML learners!");
}
</code></pre>

<button type="button">Show message</button>
```

A button inside a form defaults to submit behaviour in common HTML usage; specify type explicitly to avoid accidental form submission. Use JavaScript to implement the action.

Use pre and code together when presenting code blocks. Use code alone for short inline code fragments.

14. Accessibility and ARIA Basics

Accessibility means making web content usable by people with different abilities and by different assistive technologies. The source presentation introduces WCAG's POUR principles: Perceivable, Operable, Understandable, and Robust.

Tag / element	Purpose and usage
Perceivable	Information and interface components must be presented in ways users can perceive: for example, meaningful alt text, captions, and sufficient contrast.
Operable	Users must be able to operate controls, including by keyboard, with adequate time and without keyboard traps.
Understandable	Content, labels, navigation, and error messages should be clear and predictable.
Robust	Content should use valid, well-structured markup that works with browsers and assistive technologies.

14.1 What is ARIA?

ARIA means Accessible Rich Internet Applications. ARIA attributes add accessibility information when native HTML does not fully express a custom interface. The first rule is to use a native HTML element whenever one provides the required behaviour.

Tag / element	Purpose and usage
role	Identifies the role of a custom element, such as role="region" or role="dialog".
aria-label	Provides an accessible name directly.
aria-labelledby	Uses the text of another element as the accessible name.
aria-expanded	Communicates whether a controlled disclosure/section is expanded.
aria-controls	Identifies the element controlled by the current control.
hidden	Hides an element from rendering and the accessibility tree while the attribute is present.

```
<button id="shippingButton"
      aria-expanded="false"
      aria-controls="shippingPanel">
   Shipping information
</button>

<div id="shippingPanel"
    role="region"
    aria-labelledby="shippingButton"
    hidden>
   Free shipping on eligible orders.
</div>
```

If JavaScript opens or closes the panel, it must keep aria-expanded and the hidden state synchronized. Do not add ARIA roles to native controls unnecessarily.

14.2 Example JavaScript state update

```
const button = document.querySelector("#shippingButton");
const panel = document.querySelector("#shippingPanel");

button.addEventListener("click", () => {
  const isExpanded =
    button.getAttribute("aria-expanded") === "true";

  button.setAttribute("aria-expanded", String(!isExpanded));
  panel.hidden = isExpanded;
});
```

15. Keyboard Navigation and Inclusive Design

A well-structured page should be usable without a mouse. Keyboard users commonly move through interactive elements with Tab and activate controls with Enter or Space, depending on the element.

15.1 Skip link

```
<a class="skip" href="#main">Skip to content</a>

<header>...</header>
<main id="main" tabindex="-1">
  <h1>Page content</h1>
</main>
```

A skip link lets keyboard users bypass repeated navigation and move directly to the main content.

15.2 Focus styling

```
.skip {
  position: absolute;
  left: -999px;
}
.skip:focus {
  left: 8px;
  top: 8px;
}
:focus-visible {
  outline: 3px solid #E44D26;
  outline-offset: 2px;
}
```

- Tab order follows the order of focusable elements in the document. Keep DOM order logical.
- Avoid positive tabindex values such as tabindex="1" because they can create confusing focus sequences.
- Provide a visible focus indicator; do not remove outlines without an accessible replacement.

15.3 Inclusive design checklist

- Write meaningful alternative text for informative images.
- Use empty alt text for decorative images.
- Label every form control.
- Do not rely on colour alone to communicate meaning.
- Provide captions for relevant video/audio content.
- Respect reduced-motion preferences.
- Test at 200% zoom and on narrow screens.
- Check that all interactive features work using a keyboard.

```
@media (prefers-reduced-motion: reduce) {
  * {
    animation: none !important;
    transition: none !important;
  }
}
```

The source slide gives a contrast example and notes a 4.5:1 contrast ratio for normal text at WCAG AA. Actual contrast requirements depend on text size and content; check the applicable WCAG criterion when evaluating a page.

16. Metadata, SEO, and Structured Data

16.1 Metadata in the head

```
<head>
  <meta charset="utf-8">
  <meta name="viewport"
      content="width=device-width, initial-scale=1">
  <title>Semantic HTML Guide | University</title>
  <meta name="description"
      content="Learn semantic HTML, accessibility, and page structure.">
  <link rel="stylesheet" href="app.css">
  <link rel="canonical" href="https://example.edu/html-guide">
  <meta name="theme-color" content="#17365D">
</head>
```

Tag / element	Purpose and usage
<title>	Title shown in the browser tab and used as a search/bookmark title.
<meta charset="utf-8">	Declares the character encoding.
meta name="viewport"	Controls the layout viewport on mobile devices; do not prevent users from zooming.
meta name="description"	Provides a concise page summary that may be used as a search-result snippet.
<link rel="stylesheet">	Associates an external CSS stylesheet.

<link rel="canonical">	Identifies the preferred URL for a page when duplicate or similar URLs exist.
meta property="og:title" / "og:image"	Open Graph metadata can influence the title and image shown in social sharing previews.
meta name="theme-color"	Suggests a browser UI theme colour on supported devices.

16.2 Script loading

```
<script src="app.js" defer></script>
```

The defer attribute downloads an external classic script while the HTML is being parsed and executes it after parsing, helping avoid parser-blocking execution.

16.3 SEO-oriented HTML structure

- Use a clear page title and a useful meta description.
- Use a logical heading hierarchy, beginning with the page's main heading.
- Write descriptive link text.
- Provide meaningful alt text for informative images.
- Use semantic landmarks and meaningful page content.
- Build fast, mobile-friendly pages.
- Use structured data only when it accurately describes the page.

16.4 JSON-LD structured data example

```
<script type="application/ld+json">
{
 "@context": "https://schema.org",
 "@type": "Article",
 "headline": "Semantic HTML",
 "author": {
  "@type": "Person",
  "name": "A. Dev"
 },
 "datePublished": "2026-11-04"
}
</script>
```

JSON-LD is structured data embedded in a script element. The example describes an Article. The values must match the actual page; do not add misleading structured data.

17. Integrated Practical Exercise: Build a Semantic Course Page

Task: Create a simple course information page using the tags introduced in these notes.

Requirements

23. Create a valid HTML5 document with lang, charset, viewport, and title.

24. Add a header with the course name and a nav containing at least three links.
25. Use one main element and a clear h1 heading.
26. Add sections for course overview, learning outcomes, and weekly topics.
27. Use an ordered or unordered list for topics.
28. Add a table showing week number, topic, and practical activity.
29. Add a figure with an image and figcaption.
30. Add an aside with related resources.
31. Add a contact form with labels, email input, select, textarea, and submit button.
32. Add a footer with contact information and a time element.
33. Check keyboard navigation, alt text, labels, and heading order.

Starter structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Front End Development Frameworks</title>
</head>
<body>
  <header>
    <h1>Front End Development Frameworks</h1>
    <nav aria-label="Primary navigation">
      <a href="#overview">Overview</a>
      <a href="#topics">Topics</a>
      <a href="#contact">Contact</a>
    </nav>
  </header>

  <main>
    <section id="overview">
      <h2>Course overview</h2>
      <p>Write a short course description here.</p>
    </section>

    <section id="topics">
      <h2>Topics</h2>
      <ul>
        <li>HTML</li>
        <li>CSS</li>
        <li>JavaScript</li>
      </ul>
    </section>

    <section id="contact">
      <h2>Contact</h2>
      <form action="/contact" method="post">
        <label for="studentEmail">Email</label>
        <input id="studentEmail" name="email" type="email" required>
        <button type="submit">Submit</button>
      </form>
    </section>
  </main>

  <footer>
    <p>Course information</p>
  </footer>
</body>
</html>
```

18. Revision: Tag Coverage Checklist

Area	Tags, attributes, and concepts covered
Document structure	DOCTYPE, html, head, title, body, meta, link, script
Headings and text	h1-h6, p, strong, em, b, i, blockquote, br, hr
Links	a; href, target, rel, download, mailto:, tel:, fragment links
Images	img; src, alt, width, height, loading; figure, figcaption, picture, source, srcset
Lists	ul, ol, li, dl, dt, dd; start
Tables	table, caption, thead, tbody, tfoot, tr, th, td; scope, colspan, rowspan
Semantic structure	header, nav, main, article, section, aside, footer, time
Generic containers	div, span
Embedded content	iframe; src, title, width, height, loading, sandbox, allowfullscreen; obsolete frame/frameset
Forms	form, fieldset, legend, label, input, select, option, textarea, button; action, method, id, name, required, autocomplete
Input types	text, email, password, number, date, checkbox, radio, file, range
Media and disclosure	audio, video, track, source, details, summary
Code and controls	pre, code, button
Accessibility	role, aria-label, aria-labelledby, aria-expanded, aria-controls, hidden, tabindex, focus-visible
Metadata and SEO	description, viewport, canonical, Open Graph title/image, theme-color, JSON-LD

19. Short Assessment Questions

34. What is the difference between an HTML tag, an element, and an attribute?
35. Why is <!DOCTYPE html> included at the beginning of an HTML document?
36. Differentiate head and body.
37. When should strong be used instead of b?
38. What is the difference between an absolute URL and a relative URL in an anchor element?
39. Why should every informative img element have meaningful alt text?
40. Differentiate ul, ol, and dl.
41. What is the purpose of caption, th, and scope in a table?
42. Differentiate section, article, and div.

43. Why should a page use main and nav landmarks?
44. What is the purpose of the title attribute on an iframe?
45. Differentiate GET and POST form methods.
46. Why must every form control have a label?
47. What are the four WCAG POUR principles?
48. What is the first rule when considering ARIA?
49. What is the purpose of a skip link?
50. What does the viewport meta element do?
51. What is JSON-LD used for?

20. Important Teaching Notes and Source Boundaries

The uploaded presentation covers semantic HTML, accessibility, page landmarks, core content tags, forms, iframes, ARIA, keyboard navigation, document metadata, SEO, and JSON-LD. These notes expand the examples into a classroom sequence. Some topics in the presentation are brief overview items rather than full tutorials (for example, detailed responsive srcset/sizes selection, complete form processing on a server, CSP configuration, and advanced ARIA patterns). Those topics are identified as extensions rather than represented as fully taught by the source slides.

The presentation also contains a few compressed examples. The lecture notes use syntactically complete examples for teaching, while retaining the source's intended concepts. HTML alone provides structure and native behaviour; CSS controls presentation, and JavaScript adds application behaviour.